プログラマのための

データベース入門

株式会社 治郎吉商店

白石 亘

目次

はじめに....3
データを保存する....4
ファイルに保存する....4
データベースを使う....4
リレーショナルデータベース....5
SQL....5
アプリケーションのタイプ....6
スタンドアロン....6
Web アプリケーション....7
Web サービス....8
利用者の範囲と開発対象....9
スタンドアロン....9
開発ツール....11
モデリング....12
まず表形式にする....12
キーと外部キー....13
正規化と冗長....14
よくあるテーブル....15
１つの表....15
１対１．．N....15
１対０．．１....15
０．．N 対０．．N マッチングテーブル....15
キーバリューテーブル....15
シリアル番号テーブル....15
モデリングの実際....16
テーブル名....16
フィールド名....16
よくあるフィールド....16
外部キーフィールドの名称....17
プログラム内のフィールド名....18

はじめに

ソフトウェア開発に携わる方であれば、業務用アプリケーションを開発する機会があると思います。そんなときに役立つのがデータベースについての知識です。

業務用アプリケーションではデータを保存する機能が求められます。データはファイルに保存しますが、構造化されたデータを保存して取り出すには大量のプログラミングが必要になります。データベースを使えばこれらのプログラミングが簡単になります。

本書では、データベースの中でも特にリレーショナルデータベースについて説明します。リレーショナルデータベースはビジネス用途で多く使われていますので、これを知っておくことは何かと便利だと思います。

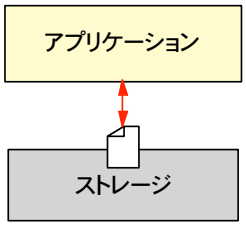
データを保存する

ファイルに保存する

例えば C 言語でファイルに何かを書き出すには次のようなプログラムを記述します。

```
fopen // ファイルを開く
fwrite // 内容をファイルに出力
fclose// ファイルを閉じる
```

ファイルを開いて、何か書き出して、閉じる、のですが、問題は fwrite で書き出す内容です。構造化されたデータ（レコード）を保存する場合は、それぞれのデータの長さや区切り方などを決めておかなくてはなりません。

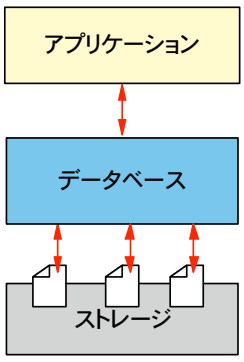


データベースを使う

例えば SQL データベース（SQL については後述）では何かを保存するプログラムは次のようになります。

```
connect // データベースに接続
insert // データベースにレコードを挿入
disconnect // データベースとの接続を解除
```

ここではファイルにどのように保存するかを意識する必要は無く、ひとかたまりのデータをデータベースに insert（挿入）するイメージになります。最終的には fwrite でファイルに保存するのですが、そこはデータベースがやってくれます。



リレーショナルデータベース

データベースには主に次のようなものがあります。

1）表計算ソフトで表現されるような、２次元の表で表されているデータ

2）CAD のように異なる形式のオブジェクトが集まっているデータ

3）長い文章のようなストリーム形式のデータ

4）音楽、動画のような時間の概念が入ったストリーム形式のデータ

ここでは、1) の表形式のデータについて説明します。複数の表形式のデータを格納するデータベースをリレーショナルデータベース（RDB - Relational Database）と呼びます。

リレーショナルデータベースを管理するための仕組みをリレーショナルデータベース管理システム（RDBMS - Relational Database Management System）と呼びます。表形式の格納場所に、データを保存（挿入）したり、保存されているデータを修正（更新）したり、削除する機能と、検索条件を与えてデータを取り出す機能が備わっています。

表のことをテーブル、１行のデータをレコード、列の構成要素（１つのセル）をフィールドと呼びます。次に説明する SQL ではテーブルは表、レコードは行、フィールドは列と呼びます。

SQL

リレーショナルデータベースを操作する方法の一つに「SQL（Structured Query Language）」があります。SQL は日本語ではエスキューエルと読みますが、英語ではシークルと発音します。プログラムにあたる構文を「SQL 文」と呼びます。

この構造化された照会言語はインタープリタとして実装されます。アプリケーション側で構文が作られて、データベースサーバまたはデータベースエンジンに転送され、サーバ側で構文が解析されて実行イメージが作られて、データベースに対して実行されます。実行された結果、データベースは結果を返します。select 文であれば、結果は選択リストになります。選択リストはデータベース側に用意されクライアントからフェッチ要求（転送命令）されたときにようやくクライアントに転送されます。

主に次のような構文があります。

- **select 文**
- **insert 文**
- **update 文**
- **delete 文**

アプリケーションのタイプ

アプリケーションにはデータへのアクセス方法によって、次のようなタイプ（型）があります。

1）スタンドアロン

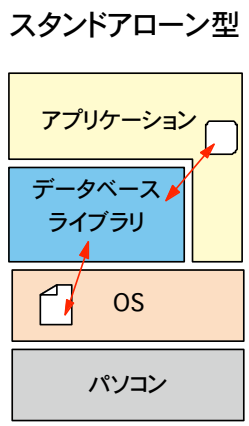
2）クライアント / サーバ

3）Web アプリケーション

4）Web サービス

スタンドアロン

単独で実行されて 1 人のユーザが単独で利用するアプリケーションをスタンドアロンアプリケーションと呼びます。データはファイルに保存されて、その利用者しかアクセスすることができません。



クライアント / サーバ

データを複数のパソコンで共有するにはデータをサーバに置きます。このサーバをデータベースサーバと呼びます。利用者が操作するソフトをクライアントアプリケーションと呼びます。

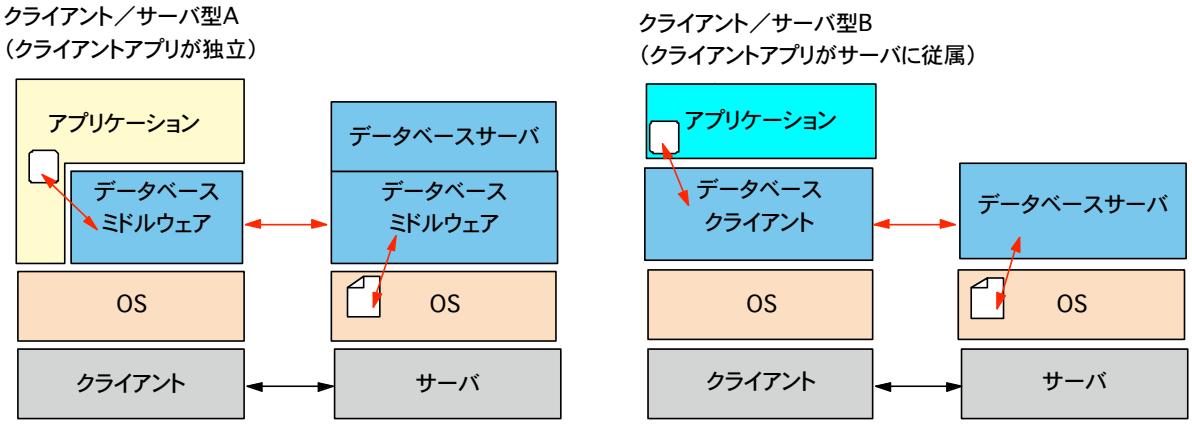
データを保存する場合、クライアントアプリケーションはリモートのデータベースにデータを送信します。データベースサーバはデータを受信してファイルに保存します。データを取り出す場合、クライアントアプリケーションから検索命令が送信されます。命令を受信したデータベースサーバは該当するデータ群を返信します。

このようにデータベースサーバとクライアントアプリケーションが通信しながら動作するアプリケーションをクライアント／サーバ型と呼びます。

クライアント / サーバ型のアプリケーションは次の 2 種類に分けることができます。

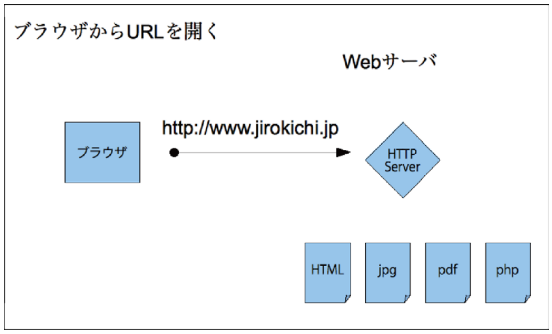
A）クライアントアプリケーションとデータベースサーバがそれぞれ独立

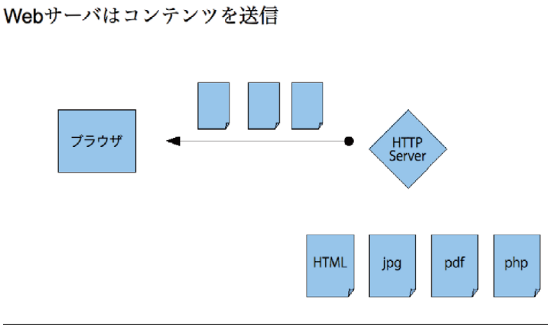
B）クライアントアプリケーションはデータベースサーバと同じベンダーのツールが必要



Web アプリケーション

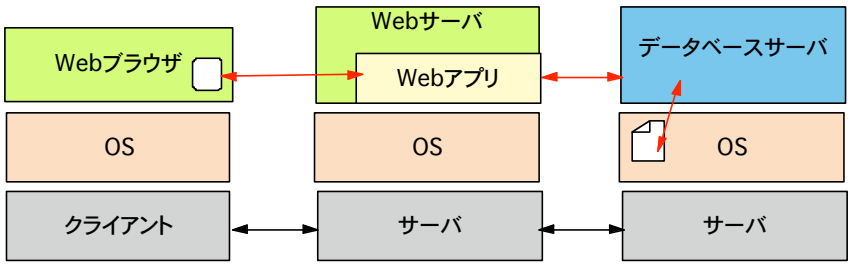
Web アプリケーションでもデータベースが使われていることはよく知られています。Web アプリケーションの場合、利用者は主に Web ブラウザを使います。Web ブラウザからの要求は、Web サーバが受けて、Web アプリケーションに渡されます。Web アプリケーションは要求を解析して、必要であればデータベースサーバにアクセスします。Web アプリケーションは応答として HTML ファイルを返します。





Web アプリケーションとデータベースサーバはクライアント / サーバの関係にあります。また、Web ブラウザと Web サーバもクライアント / サーバの関係にあります。

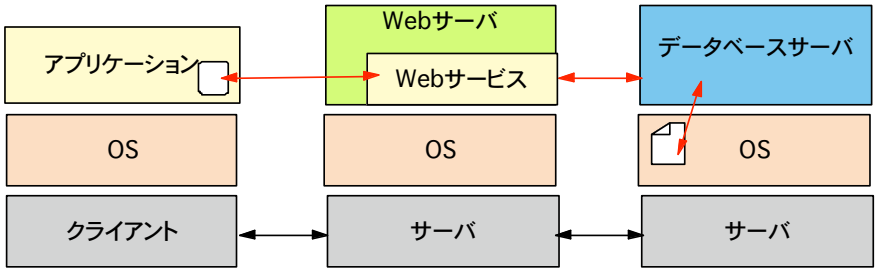
Webアプリケーション型
（Webアプリとデータベースがクライアント/サーバの関係）



Web サービス

Web ブラウザを使わずに、専用のクライアントアプリケーションから Web サーバにアクセスする方法もあります。クライアントアプリケーションからの要求は、Web サーバが受けて、Web サービスに渡されます。Web サービスは応答としてクライアントアプリケーションの要求するデータを返します。

Webサーバ型
（Webサービスとデータベースがクライアント/サーバの関係）

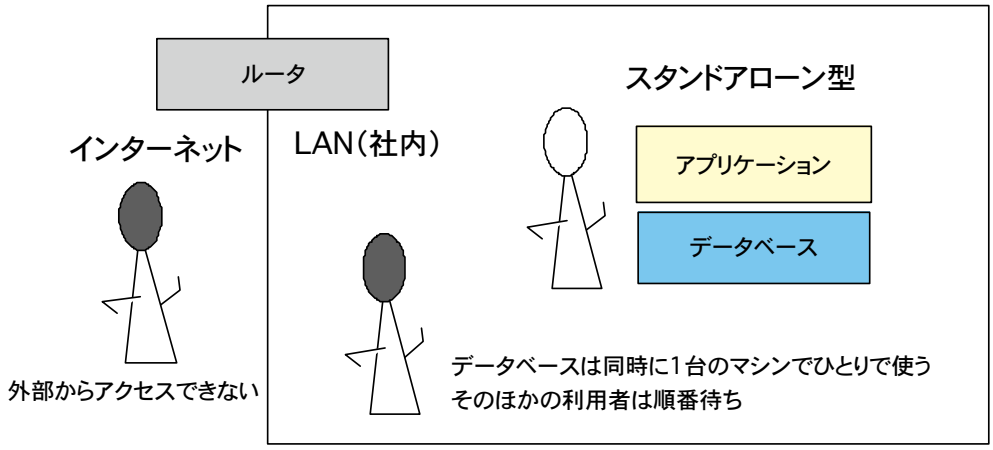


iOS デバイスのネイティブアプリからクラウドのデータベースサーバに対してデータを保存・検索するようなシステムはこの Web サービス型になります。

利用者の範囲と開発対象

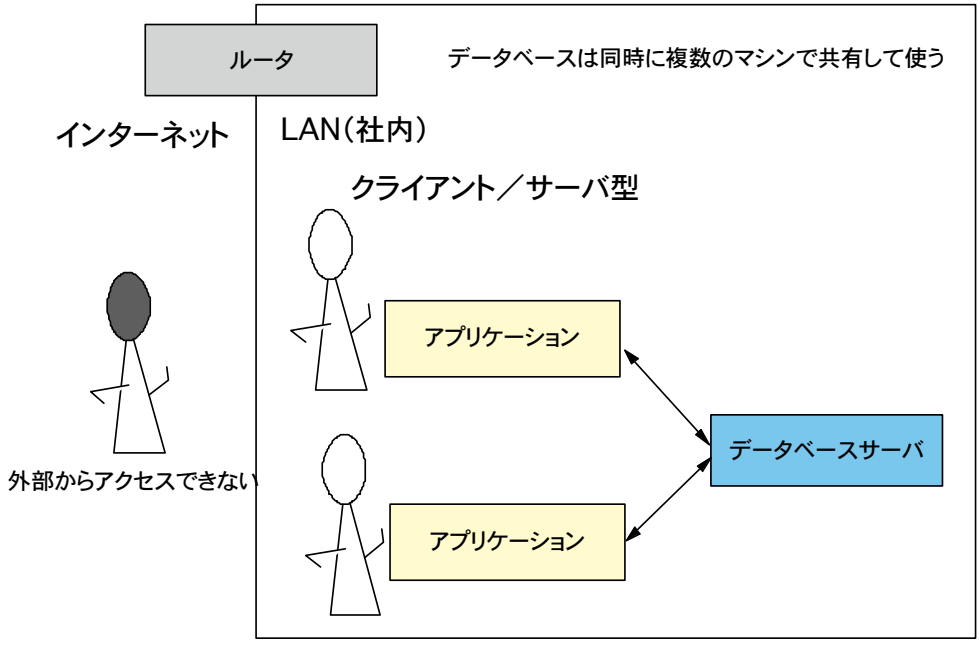
スタンドアロン

データベースはそのマシンの中にあり、同時に一人の利用者でしか使わないケースです。別の人が利用する場合はデータベースファイルをファイルサーバなどに置いてコピーして利用します。誰かが修正しているときは別の利用者にその修正内容は反映されません。



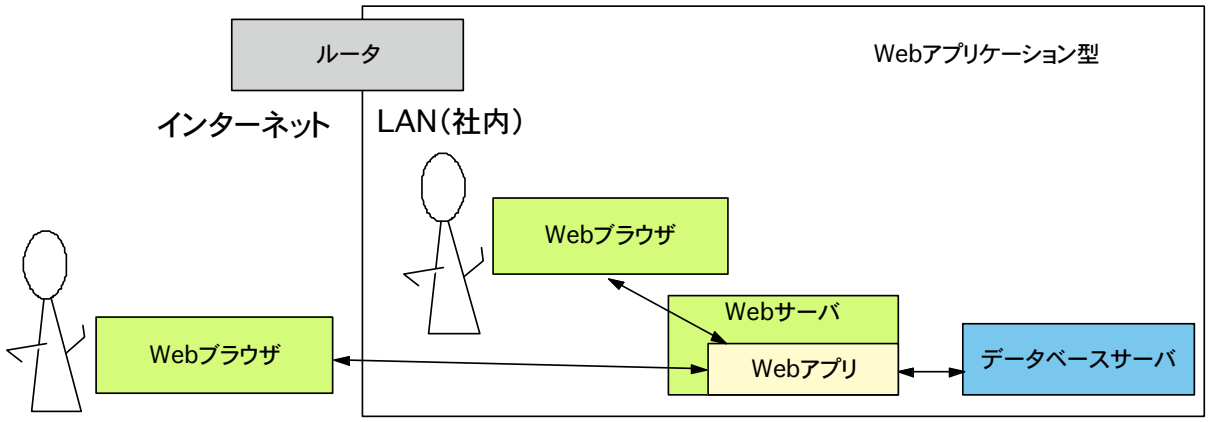
クライアント / サーバ

社内の LAN 上でデータベースを共有します。複数の利用者から同時にデータベースを利用します。誰かが修正した内容はすぐに別の利用者に反映されます。社外のインターネット経由ではデータベースを利用できません。



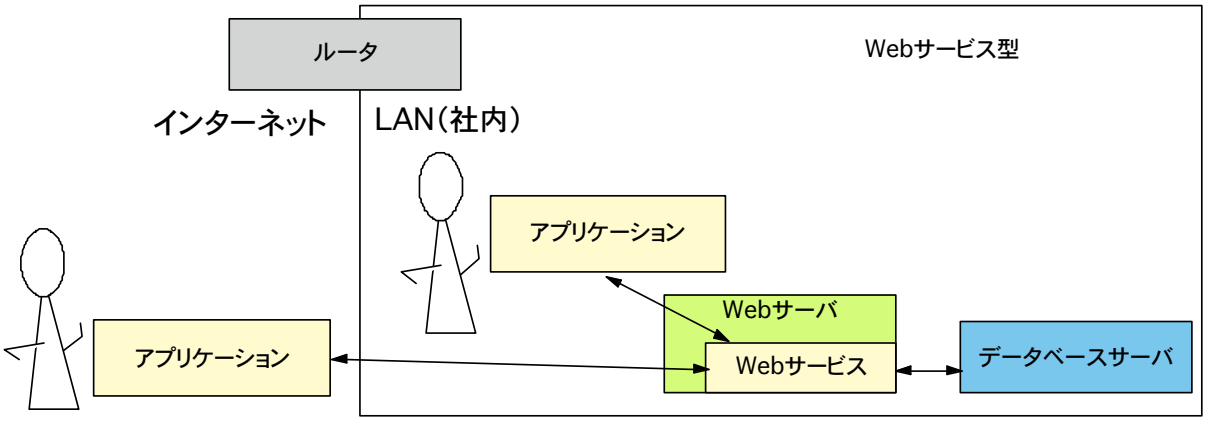
Web アプリケーション

広くインターネット上でデータベースを共有します。利用者はブラウザを使って Web サーバにアクセスして、Web コンテンツを介してデータベースサーバにアクセスします。このように利用者が使うアプリケーションがブラウザの場合は Web アプリケーションと呼びます。Web アプリケーションは Web サーバにあり、アプリケーションを配布する必要がありません。



Web サービス

広くインターネット上でデータベースを共有します。利用者は専用のネイティブアプリケーションを使ってデータベースにアクセスします。このように利用者が使うアプリケーションがブラウザではなく専用のネイティブアプリケーションの場合、サーバ側のサービスを Web サービスと呼びます。専用のネイティブアプリケーションは利用者に配布する必要があります。



ここで注意していただきたいのは、上記のアプリケーションタイプを実装するためにはそれぞれ必要となる技術が異なるということです。

スタンドアロン型の場合、アプリケーションは利用するマシンにだけ実装します。プログラミングの対象はひとかたまり（一つのプロジェクト）だけです。

クライアントサーバ型の場合、クライアント側とサーバ側のそれぞれにアプリケーションを実装します。二つのプロジェクトが必要になりそうです。

Web アプリケーション型の場合、Web サーバ側に一連のプログラムを用意します。一般的に複数のプログラムを用意することになりますが、言語や開発環境は 1 種類のことが多いでしょう。

Web サービスの場合は Web サーバ側に一連のプログラムを用意して、さらにクライアント側にアプリケーションが必要です。

どの場合もそうですが、一度でもデータの流れを作ってしまえば、あとは簡単です。

開発ツール

データベースアプリケーションを開発するときに使う開発ツールを紹介します。

スタンドアロン型	SQLite、Firebird、開発者の独自ライブラリ
クライアント / サーバ型	各種 RDBMS、FileMakerPro、4D、Microsoft Access
Web アプリケーション型	各種 Web アプリケーションサーバ、FileMakerPro、4D

RDBMS（リレーショナルデータベース管理システム）をいくつか紹介します。

MySQL	オープンソースでフリーで使えるが、商用ライセンスがある。再販するときは注意したい。
PostgreSQL	オープンソースでフリーで使える。
Firebird	オープンソースでフリーで使える。
Oracle	商用ライセンスのみ。
Microsoft SQL Server	商用ライセンスのみ。

モデリング

モデリングとは、人間が扱っている情報をデータベースの形に定義する作業です。

次のような請求書を考えてみます。



請求書

2014年 12月 10日
No.あー018517

じろ商事 御中

株式会社　治郎吉商店

251-0025
神奈川県藤沢市鵠沼石上２-５-１
カサハラビル藤沢４F
0466-54-6221　0466-54-6233

ご請求金額：　￥105,840

下記のとおりご請求いたします。

	内容	単価	数量	金額
	商品A	30,000	2	60,000
	技術サポート料	1,500	12	18,000
	コンサルティング料金	20,000	1	20,000

合　計：　￥98,000
消費税：　￥7,840
総　計：　￥105,840

備考：

振込期限：0月0日

【振込口座】
みずほ銀行　藤沢支店　普通２６４１５６６
三菱東京UFJ銀行　藤沢支店　普通１７１２７９９
三井住友銀行　藤沢支店　普通７２４２４１１
横浜銀行　鎌倉支店　普通１４８１７８２

[口座名義]　株式会社ジロキチショウテン

まず表形式にする

請求書には次のような要素があります。

1．請求先
2．請求日
3．発信元

4．請求金額（内容の合計金額）

5．内容（商品名、数量、単価、金額）

請求書を表形式にすると次のようになります。（表１）

	A	B	C	D	E	F
1	請求先	日付	請求書番号	発信元	請求金額	内容
2	じろ商事	2014.12.10	あー018517	株式会社　治郎吉商店	105,840	商品A　300,000 x 2 = 300,000 技術サポート料　1,500 x 12 = 18,000 コンサルティング料金　20,000 x 1 = 20,000
3						

これをテーブルに格納しやすい形式、並べ替え可能な形に変形させると次のようになります。（表２）

	A	B	C	D	E	F	G	H
1	請求先	日付	請求書番号	発信元	内容	単価	数量	金額
2	じろ商事	2014.12.10	あー018517	株式会社　治郎吉商店	商品A	300,000	2	60,000
3	じろ商事	2014.12.10	あー018517	株式会社　治郎吉商店	技術サポート料	1,500	12	18,000
4	じろ商事	2014.12.10	あー018517	株式会社　治郎吉商店	コンサルティング料金	20,000	1	20,000
5								

このようにテーブルに格納しやすい形、更新しやすい形にしていくことを正規化と呼びます。表２の状態を第一正規形と呼びます。

キーと外部キー

あるレコードを修正・削除するときは表の中でそのレコードを特定しなくてはなりません。そのためには表の中でユニークな値を持つ ID フィールドが必要になります。この役目を持つフィールドを「キー」と呼びます。前の図ではレコードには ID フィールドがありませんでした。ID フィールドを追加すると次のようになります。（表３）

	A	B	C	D	E	F	G	H	I
1	ID	請求先	日付	請求書番号	発信元	内容	単価	数量	金額
2	1001	じろ商事	2014.12.10	あー018517	株式会社　治郎吉商店	商品A	300,000	2	60,000
3	1002	じろ商事	2014.12.10	あー018517	株式会社　治郎吉商店	技術サポート料	1,500	12	18,000
4	1003	じろ商事	2014.12.10	あー018517	株式会社　治郎吉商店	コンサルティング料金	20,000	1	20,000
5									

この表をさらに正規化すると次のようになります。（表４）

請求書テーブル				
ID	請求先	日付	請求書番号	発信元
11	じろ商事	2014.12.10	あー018517	株式会社　治郎吉商店

請求内容テーブル					
ID	内容	単価	数量	金額	請求書ID
1001	商品A	300,000	2	60,000	11
1002	技術サポート料	1,500	12	18,000	11
1003	コンサルティング料金	20,000	1	20,000	11

テーブルが二つに別れ、請求内容テーブルにも ID が追加されました。請求内容テーブルには、内容に関連する請求書の ID を格納する「請求書 ID」というフィールドが追加されています。この請求内容テーブルの請求書 ID のように、別テーブルのレコードを特定するためのフィールドを「外部キー」と呼びます。

正規化と冗長

リレーショナルデータベースでは、正規化によってデータ構造は無駄の無い形になっていきます。請求明細レコードを特定するためのキーが付いた状態の第一正規形から始めて、商品を別テーブルにしたり、請求書と請求内容を 1 対 N の別テーブルにしたり、と続きます。

しかし正規化を進めていくと、実務と合わないケースも出てきます。請求内容の商品を例にして説明しましょう。例えば商品名と単価を別テーブルにして、請求内容には商品コードだけを持たせるようにします。（表 5）

請求書テーブル				
ID	請求先	日付	請求書番号	発信元
11	じろ商事	2014.12.10	あー018517	株式会社　治郎吉商店

請求内容テーブル				
ID	商品ID	数量	金額	請求書ID
1001	501	2	60,000	11
1002	502	12	18,000	11
1003	503	1	20,000	11

商品テーブル		
ID	商品名	単価
501	商品A	300,000
502	技術サポート料	1,500
503	コンサルティング料金	20,000

まず請求書を発行して、そのあとで商品名を変更した場合について考えます。以前の請求書の商品名を「商品 A」名前変更後の商品名が「商品 B」だとします。請求内容に商品名を持たせないで商品 ID だけを保存していたとします。商品 A の時にいくつか売れて請求書を作成します。そのあとで商品名が商品 B に変更された場合、過去の請求内容を表示させると商品名は「商品 B」になります。このようになってほしい場合となってほしくない場合があります。なってほしくない場合は、請求時の商品名を保存しておくことになります。このようなことを冗長と呼びます。

よくあるテーブル

１つの表

ID をもった一つの表です。マスター的なデータを持つ表と、仕事の内容を保存するためのトランザクション的な表があります。案件によってははじめマスターデータだと思っていた表が実はトランザクション的だったりして、マスターかどうかは問題にする必要はありません。

１対１.. N

ある表の１レコードに対して、別の表の複数レコードが関係している、という構造です。請求書と請求内容のような関係です。１対 N 構造と呼びます。N 側のテーブルが１側のレコード ID を保持します。画面表示を意識してフォームアンドディテールと呼ぶこともあります。

１対０.. １

ある表の１レコードに対して、別の表の１件または０件のレコードが関係している、という構造です。１対１構造と呼びます。ある表のレコードの属性を別の表に持たせる場合です。多数の属性フィールドがあったり、レコードによっては属性がすべて空白だったりすると、この部分を別表にしたくなります。ある表にそれらすべてのフィールドを持たせようとすると空白が多くなりすぎるからです。たとえば人材テーブルから配偶者の情報は別テーブルにするとか。

０.. N 対０.. N マッチングテーブル

ある２つの表を結びつけるためのテーブルです。たとえば人材テーブルのある人材に、特技を登録する場合を考えます。特技は特技テーブルに登録しておいて選択することにします。この場合ある人材には複数の特技が登録されますし、ある特技には複数の人材が登録されるはずです。また特技がない人や誰にも登録されないと区議があったりします。

キーバリューテーブル

キーを渡すとバリュー（値）が返ってくる、というプログラムに使うテーブルです。プレファレンス情報のようなパラメータを保存するのに向いています。この種の情報は外部テキストに保存することもできますが、一般にデータベースに保存した方が便利です。ファイルに保存するとアクセス権の問題にぶつかりやすくなります。

シリアル番号テーブル

ほかの表に ID を供給するためのテーブルです。各テーブルの最終 ID を保持しています。

モデリングの実際

ある現場で行われているモデリングについて紹介しましょう。ネーミングルールとデータ構造の参考例です。一般に多くのプロジェクトではネーミングルールを決めていると思いますが、ここは１例として参考にしてください。

テーブル名

テーブル名には英数字とアンダーバー（_）を使います。英字は大文字にします。ツールによっては大文字小文字を区別しない環境があります。このためＣ言語などでよく使う１文字目だけを大文字にして単語を区切る方法はお勧めしません。単語を区切る場合はアンダーバー（アンダースコアとも呼ぶ）を使いましょう。

テーブル名には２文字〜５文字の略称を決めておきます。たとえば人材テーブルを「JINZAI」とすると、略称は「JI」、特技テーブルを「TOKUGI」とすると略称は「TO」にします。

マッチングテーブルの場合は、たとえば人材テーブルと特技テーブルのそれぞれの ID を保持するためのマッチングテーブルは名前も略称も「TOxJI」にします。

フィールド名

フィールド名には英数字とアンダーバー（_）を使います。業務上のキーワードの場合、定義に曖昧さが無く、関係者全員の概念モデルがが一致しているものは、その名称を尊重しましょう。たとえば契約開始日は「START_DATE」や「CONTRACT_START_DATE」ではなく「KEIYAKU_KAISHIBI」にします。こうしておくと画面を作成するときに楽になります。

テーブル略称を接頭辞にしてフィールド名に付けるようにします。たとえば人材 ID は「JI_ID」、人材名は「JI_NAME」です。

よくあるフィールド

いつも決まって用意することになるフィールドがあります。これらは何も考えずに作っておくと時間の節約になります。まず次は必須のフィールドです。

ID	レコードを識別するための倍長整数フィールド。インデックスを付ける。
NAME	名前。レコードを代表する文字列フィールド。
DEL_FLAG	削除フラグ。レコードが削除されているかどうかを判定するための倍長整数フィールド。特に理由がない場合はレコードは物理的に削除しない。
CREATE_DATEMARK	レコード作成日を示す文字列フィールド。YYYYMMDDHHMMSSmmm の１７文字。日付と時刻を考慮したソートに使えるほか、別のデータベース管理システムにデータ移行する場合に安心。
UPDATE_DATEMARK	レコード更新日を示す文字列フィールド。YYYYMMDDHHMMSSmmm の１７文字。日付と時刻を考慮したソートに使えるほか、別のデータベース管理システムにデータ移行

する場合に安心。

INPUT_DATE　入力日。これは日付型で保存しておきます。時刻も保存しておくとよいかもしれません。

そのほかあると便利なフィールドを紹介します。

SORT_ORDER　並び順。特にマスター的な選択肢となるようなデータには用意しておくと良いでしょう。デフォルト値は作成順でしょうから、ID を 10 倍した整数値を入れておきます。

SHOW_HIDE　表示・非表示フラグ。削除はしないけど見えていると邪魔、というデータはあります。

BIKOU　備考。ユーザが入力するデータ構造には備考欄が必須だと思います。「REMARK」とか「NOTE」「MEMO」でもよいでしょう。

人材テーブルの場合は次のようなテーブル構造になります。

JI_ID	**人材 ID**
JI_NAME	**名前**
JI_DEL_FLAG	**削除フラグ**
JI_CREATE_DATEMARK	**作成デイトマーク**
JI_UPDATE_DATEMARK	**更新デイトマーク**
JI_INPUT_DATE	**入力日**
JI_REMARK	**備考**

人材レコードは人材データベースシステムの主役ですから、並び順や表示・非表示フラグは不要と考えました。これらに加えて、誕生日、性別、住所、連絡先など業務上必要となるフィールドを追加していきます。

外部キーフィールドの名称

たとえば人材テーブルに経歴テーブルが関連しているとします。一人の人材に複数の経歴があります。人材に関連していない経歴はないものとします。経歴テーブルは次のようになります。

KE_ID	**経歴 ID**
KE_JI_ID	**人材テーブルのレコードを特定する外部キー**
KE_NAME	**経歴名**
KE_KAISHIBI	**経歴開始日**
KE_SHUURYOUBI	**経歴終了日**
KE_DEL_FLAG	**削除フラグ**
KE_UPDATE_DATEMARK	**更新デイトマーク**
KE_REMARK	**備考**

ある人材の経歴を画面に表示する場合、人材 ID で経歴テーブルの人材 ID を検索して、該当したレコードをデータベースから取り出します。逆に人材テーブルに経歴 ID を持たせることはないのでしょうか。答えは「ない」です。人材テーブルをいくら見ても関連する経歴がある、という情報はありません。経歴があるかどうかを知るためには人材 ID で経歴テーブルをクエリーすればいいのです。

プログラム内のフィールド名

テーブルとフィールドを上記のようなネーミングルールにしますと、データベースシステム全体でフィールド名はユニークになります。この結果、プログラムはどうなるのでしょうか。

プログラムでは、テーブルから取り出したデータを保持するための変数名にフィールド名をそのまま使えます。変数や関数の名前を決めることに多くの時間を使っているプログラマーは多いと思います。フィールド名がすでにユニークになっていれば、プログラミングの最中に変数名を考えなくてよくなります。

同じ理由でテーブル名とフィールド名には日本語を使わない方がいいと思います。